arXiv:2411.19460v1 [cs.CV] 29 Nov 2024

Look Every Frame All at Once: Video-Ma^{2}mba for Efficient Long-form Video Understanding with Multi-Axis Gradient Checkpointing

Hosu Lee* Junho Kim* Hyunjun Kim Yong Man Ro†
Integrated Vision and Language Lab, KAIST, South Korea
{leehosu01, arkimjh, kimhj709, ymro}@kaist.ac.kr
https://ivy-lvlm.github.io/Video-MA2MBA

Abstract

With the growing scale and complexity of video data, efficiently processing long video sequences poses significant challenges due to the quadratic increase in memory and computational demands associated with existing transformer-based Large Multi-modal Models (LMMs). To address these issues, we introduce Video-Ma^{2}mba, a novel architecture that incorporates State Space Models (SSMs) within the Mamba-2 framework, replacing the attention mechanisms. This allows the LMMs to scale linearly in terms of time and memory requirements, making it feasible to handle long-duration video content. Furthermore, we enhance the memory efficiency introducing the Multi-Axis Gradient Checkpointing (MA-GC) method, which strategically manages memory by retaining only essential activations across multiple computational axes. Our approach significantly reduces the memory footprint compared to standard gradient checkpointing. Empirical analyses show that Video-Ma^{2}mba can process extensive video sequences—equivalent to millions of tokens or over two hours of continuous sequences at 1 FPS—on a single GPU. By maintaining a detailed capture of temporal dynamics, our model improves the accuracy and relevance of responses in long video understanding tasks, demonstrating substantial advantages over existing frameworks.

1. Introduction

As video data grows in scale and complexity, the demand for models capable of efficiently processing long video sequences has intensified. Transformer-based models [3, 41] have become central to sequence processing due to their effectiveness and versatility in handling complex dependencies as input frames increase. With the emergence of the Large Language Models (LLMs) era [14, 30, 31], video understanding models have entered a new phase. Thanks to the core capabilities of zero-shot learning and strong reasoning in LLMs, various Large Multi-modal Models (LMMs) [9, 25, 26] have achieved enhanced cross-modality consistency, particularly between vision and language. Accordingly, diverse video-LMMs [21, 23, 29] have been proposed to understand video content by integrating spatio-temporal information into LLMs, and achieved comparable reasoning performances to comprehend complex visual narratives and temporal dynamics.

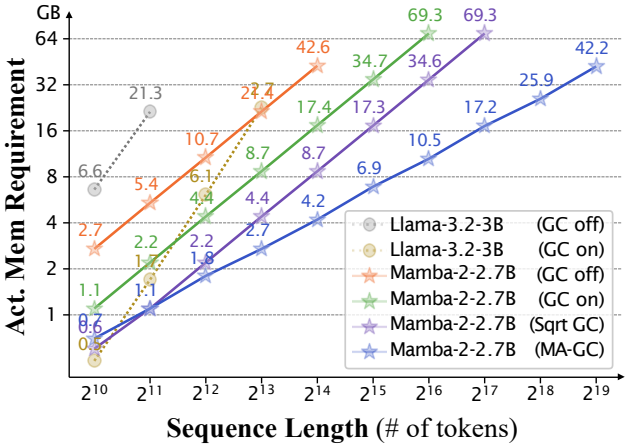


Figure 1. Memory usage comparison across sequence lengths for Mamba-2-2.7B with different checkpointing methods, demonstrating the memory-saving capability of Multi-Axis Gradient Checkpointing (MA-GC).

However, current video-LMMs face critical challenges when applied to longer video sequences integrated into the LLM structure, as the attention mechanism [42]—the core component of Transformers—incurs memory load and computational costs that scale quadratically with sequence length. This growth in resource requirements becomes prohibitive for processing extended video sequences, where sequence lengths often exceed 128K tokens (approximately 400 frames), resulting in significant inefficiencies in memory consumption and computational load.

To address the challenge for the long video understand-

*Equal contribution. † Corresponding author.

ing, various strives have been evident with several different approaches: (i) sparse and uniform sampling (*e.g.,* typically 8 or 16 frames) to reduce the number of input video frames into LMMs, which is the most common method [20, 23, 24] and (ii) memory-augmented generation [16, 39] that stores long-term visual information and later access to the memory bank. More recently, Zhang *et al.* [48] have proposed a method for transferring long context by employing RoPE-based [40] frequency extension in LM backbones within existing memory limits, which enables the models to process more visual tokens for the longer video sequences. Despite such efforts, the exponential growth in memory usage with increasing sequence lengths imposes fundamental limitations on the amount of information that can be fed into the model. Consequently, when the model is prompted to respond based on missing information (or frames), it often generates responses that are irrelevant and disconnected from the facts or user queries.

Here, the core difficulty lies in the quadratic time and space complexity of the Transformer's attention mechanism, which restricts efficient processing of lengthy video sequences. To enable more practical scalability when handling longer sequences, the fundamental solution is on shifting from quadratic complexity to linear complexity, facilitating more scalable processing for the video data. To do so, in this paper, we introduce **Video-Ma^{2}mba**, specifically designed to handle extremely long video sequences all at once. We substitute the Transformer-based LLMs [8, 41] to Mamba-2 [11] structure, utilizing State Space Models (SSMs) [15] as a replacement for the attention mechanism. This allows our framework to not only preserve the effectiveness of sequence processing but also enhance memory efficiency, thereby achieving linear time and space complexity with respect to the sequence length.

In addition to the architectural changes, to push the boundary of memory utilization within Mamba-2 architecture, we present a new **Multi-axis Gradient Checkpointing** (MA-GC) method. Specifically, the Gradient checkpointing (GC) [6] strategically saves selected activations throughout the computational graph, allowing only partial activations to be re-computed during the backward pass (therefore, widely used for managing the substantial memory demands of extensive attention layers in Transformer-based LLMs). On the other hands, due to the nature of Mamba structure, which belongs partially observed Markov models where the hidden state progresses over time according to a Markov process rather than dense interaction models (*e.g.,* Transformer), we can implement another axis for GC in the sequential direction by selectively retaining only those sequence-wise activations necessary for backpropagation. The MA-GC reduces memory usage from the $O(L \cdot S)$ complexity of the original Mamba-2 to $O(S)$ by applying GC in multiple directions, which allows Video-Ma^{2}mba to process full video sequences at 1-second intervals without the need for frame sampling. Our empirical analyses in Fig. 1 indicate that the proposed method performs effectively in handling extended sequences, successfully processing sequence lengths in the millions on a single GPU, corresponding to over 2-hours of continuous video input at a frame rate of 1 FPS. By observing each frame at regular intervals, our approach captures more comprehensive temporal information than uniform sampling, providing improvements in model responses and memory efficiency.

Through extensive experiments and computational analyses on MA-GC, we demonstrate that Video-Ma^{2}mba efficiently manages resource demands, effectively breaking the quadratic memory growth for handling long sequence.

Our contribution can be summarized into three-fold:

- We propose Video-Ma^{2}mba, a new multi-modal framework designed to handle extensively long video sequences without losing frame information, by replacing the Transformers with Mamba-2 architecture.
- To significantly enhance memory utilization within our framework, we introduce the Multi-axis Gradient Checkpointing strategy. Our strategy selectively stores activations in a bi-axis direction, effectively reducing the space complexity to $O(S)$.
- Through extensive evaluation and analyses, we corroborate that our framework can efficiently process sequence lengths in the millions, corresponding to up to 2-hours of video sequence at 1 FPS with competent performance.

2. Related Work

2.1. Context Extension Methods

Training models with extended sequence lengths has become increasingly challenging due to the computational and memory demands associated with scaling sequence lengths. The standard Transformer models [3, 42] struggle with the quadratic complexity of attention mechanisms, which quickly becomes infeasible for long sequences. To address these limitations, various methods have been proposed to extend the context length either during or after pre-training.

One of the common approaches is to generalize knowledge learned over shorter sequences to longer sequences [10, 35]. However, this approach can lead to issues in positional extrapolation, as positional encodings trained on shorter contexts may not generalize well to longer contexts. Techniques such as Rotary Position Embedding [40] and Position Interpolation [5] mitigate this by modifying the positional embedding, making it better suited for extrapolation beyond the training range. Additionally, ALiBi [35] has applied attention biases directly, improving context extension without a strict reliance on positional encodings.

An alternative approach is leveraging Structured State Space Models (SSMs) [15], which inherently achieve lin-

ear time and space complexity with respect to the sequence length. This efficiency stems from the intrinsic properties of SSMs, allowing for effective training on extended contexts and facilitating long-sequence learning without the prohibitive memory costs.

2.2. Long Video Understanding with LMMs

Long video understanding presents unique challenges due to the need to capture dependencies across extended sequences while managing high memory consumption. The core challenges in handling long videos are memory limitations and finite context lengths. Several studies [18, 50] have addressed this by sparsely sampling video frames (typically 8 or 16 from the video instance rather than using dense fps-based sampling), or by employing token compression to reduce data to a more manageable size [22, 29]. Additionally, memory-augmented approaches [16, 39] have been proposed to store relevant information beforehand and recall explicit knowledge when generating responses.

While such methods are simpler to implement and effective for managing memory, they risk missing critical details in long video content. Our approach addresses these limitations by utilizing full-frame sequences at 1 FPS, ensuring comprehensive temporal representation without relying on sparse sampling and achieving memory efficiency.

2.3. Gradient Checkpointing Techniques

The gradient checkpointing (GC) is a well-established method for reducing memory usage in deep learning models by selectively storing intermediate activations and recomputing them as needed during the backward pass. Initially developed to manage memory constraints in training, this approach allows models to trade off additional computation for reduced memory requirements.

Chen *et al.* [6] have introduced fundamental checkpointing techniques applicable across deep networks and recurrent neural networks (RNNs). For deep networks, the technique involves segmenting the network along the layer axis, storing the outputs at segment boundaries, and recomputing the intermediate results within each segment as needed. This segmentation reduces memory requirements from $O(n)$ to $O(\sqrt{n})$, where n is the number of layers. For RNNs, a similar approach is used along the time-axis, allowing memory usage to scale sublinearly with sequence length by storing checkpoints at specific time intervals.

Our work builds on these principles by applying the GC in both the layer and sequence dimensions, enabling efficient processing of long sequences across bi-directional axes. Our approach enhances memory efficiency and is particularly well-suited for understanding long videos, where integrating both temporal and spatial contexts is crucial.

3. Video-Ma^{2}mba

Overview. We first elaborate on the distinction of the Mamba-2 architecture in handling memory efficiency, then introduce a new gradient checkpointing method that can be a key factor in extending the sequence length of Mamba-2. By seamlessly implementing our new context extension strategy during the training of Video-Ma^{2}mba, our video model can handle up to maximum 0.8M input sequence tokens and overcome current challenges in long video understanding relying on partial frame sampling.

3.1. Preliminary: Mamba-2 and Simplification

The Mamba model [15] initially has leveraged structured state space models (SSMs) to efficiently handle sequence modeling. Building on this foundation, Mamba-2 [11] represents a further advancement to scale up to larger state sizes with the concept of Structured State-Space Duality (SSD), which enhances sequence processing capabilities through time-varying state transitions and input-output mappings, thus more effective handling for sequence data. The general form of SSD in Mamba-2 can be formulated as:

$$h_t = A_t h_{t-1} + B_t x_t, \quad y_t = C_t h_t, \tag{1}$$

where $A_t \in \mathbb{R}^{N\times N}$, $B_t \in \mathbb{R}^{N\times 1}$, and $C_t \in \mathbb{R}^{1\times N}$ are state matrices that vary over time, allowing the model to adapt dynamically to different input structures. This time variance, or *selectivity*, enhances Mamba-2's flexibility in comparison to linear time-invariant SSMs. By employing time-varying matrices A_t, B_t, and C_t, Mamba-2 can be seen as a selective SSM that performs sequential updates to the hidden state h_t based on previous states and current inputs.

This selective structure is particularly advantageous in capturing sequence dynamics over longer frames, which standard Recurrent Neural Networks (RNNs) [12] with fixed parameters struggle to achieve. At the same time, Mamba-2 also shares some similarities with a certain RNN framework when non-linear activations are removed. A standard RNN with a non-linear activation function σ (*e.g.*, Tanh or ReLU) updates its hidden state as follows:

$$h_t = \sigma(Ah_{t-1} + Bx_t), \quad y_t = \sigma(Ch_t). \tag{2}$$

Here, removing the activation function σ transforms the RNN, making its structure similar to that of SSD:

$$h_t = Ah_{t-1} + Bx_t, \quad y_t = Ch_t. \tag{3}$$

Consequently, SSD can be regarded as a simplified version of RNN, where the time-varying parameters of SSD introduce a level of flexibility and adaptability that fixed-parameter RNNs lack. This dynamic modification allows SSD to effectively address challenges associated with static parameter models in handling complex temporal sequences.

3.2. Multi-Axis Gradient Checkpointing

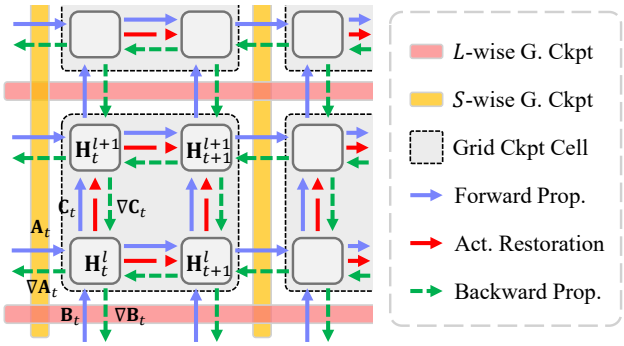
Considering that Mamba-2 follows RNN-like structure, as illustrated in Fig. 2, when processing the SSD state H_{i+1}^{l}, it only requires the prior state H_{i}^{l}, not H_{i-1}^{l}, unlike Transformers that require all previous states to calculate attention weights across the entire input sequence. Here, it is important to note that this distinction enables us to introduce an additional gradient checkpointing axis, not only along the layer direction but also uniquely along the sequence direction, which attribute to the architectural properties of Mamba-2 (whereas Transformer cannot achieve).

Our key motivation for employing bi-axis checkpointing lies in its effectiveness at managing memory demands, which enables the processing of extremely long video sequences in their entirety without needing to sample scenes partially. Here, we introduce a new GC strategy, **Multi-Axis Gradient Checkpointing** (MA-GC), that not only increases the feasible sequence length up to 2^{19} but also substantially cuts activation memory usage. While previous methods such as the $\sqrt{L}$ layer grouping [6] achieved some memory reduction by applying checkpoints every $\sqrt{L}$ layers, they were inadequate for very long sequences. In contrast, our MA-GC method applies checkpointing along both layer and sequence axes, reducing space complexity from $O(\sqrt{L} \cdot S)$ in standard GC to just $O(S)$. This significant improvement allows our model to process longer sequences more efficiently without partial frame sampling, thus supporting extended sequence lengths in understanding long video content.

Specifically, as shown in Fig. 2, our MA-GC strategy involves two checkpoint types in the forward pass for the given S sequence length and L stacked layers: (i) Layer-wise checkpoints, where layer activations are stored every l layers, and (ii) Sequence-wise checkpoints, where states across all layers are stored every s time steps. The intersecting points of these two checkpoints create *grid cells* that are essential for efficient backpropagation. Within each grid cell, activations are sequentially restored and gradients are propagated in an efficient manner. This grid-based structure facilitates selective reconstruction of states only when necessary, thereby optimizing memory usage during the computationally intensive backpropagation process. We provide detailed explanations of both the forward and backward processes in Algorithm 1 and Algorithm 2.

3.3. Analysis of Upper Bound of Memory Reduction

To understand how MA-GC reduces memory usage, we analyze its space complexity and establish an upper bound on memory savings. In a naive RNN (or similarly, in Mamba or Mamba-2) architecture with L layers and a sequence length S, backpropagation requires storing activation memory of $\Theta(L \cdot S)$ due to the need to store activations for each layer and each time step during the backward pass.



Method	*L*-dir	*S*-dir	Max Seq. Len	Active Mem.	Space Complexity
GC off	✗	✗	2^{14}	42.6 GB	$O(L \cdot S)$
GC on	✓	✗	2^{16}	17.4 GB	$O(L \cdot S)$
sqrt GC	✓	✗	2^{17}	8.7 GB	$O(\sqrt{L} \cdot S)$
MA-GC	✓	✓	2^{19}	4.2 GB	$O(S)$

Figure 2. Overview of MA-GC grid structure. Checkpoints are stored every l layers and s steps. The blue, red, and green arrows indicate forward propagation, activation restoration, and gradient propagation, respectively. This grid design optimizes memory by selectively restoring activations as needed. The below table shows comparison of checkpointing usage, maximum sequence length on 80GB VRAM, and peak activation memory in BFloat16 at sequence length 16384.

In our proposed MA-GC method, the required memory M is given by:

$$M = \underbrace{\frac{LS}{l}}_{\text{L-wise G. Ckpt}} + \underbrace{\frac{LS}{s}}_{\text{S-wise G. Ckpt}} + \underbrace{ls}_{\text{Grid Ckpt Cell}}, \tag{4}$$

where l and s represent the checkpoint intervals along the layer and sequence directions, respectively. The first term accounts for the memory needed for layer-wise checkpoints, the second term for sequence-wise checkpoints, and the third term for activations stored within each grid cell during recomputation, respectively.

Our objective is to minimize M by finding optimal values for l and s, thus maximizing memory savings. To achieve this, we employ the *Extreme Value Theorem* and *Fermat's Theorem* to identify the minimum values of a differentiable function on a closed interval.

Theorem 1 (Extreme Value Theorem) *If f is continuous on a closed interval $[a, b]$, then f achieves both maximum and minimum values on $[a, b]$.*

Theorem 2 (Fermat's Theorem) *If f is differentiable on an open interval (a, b) and has a local extremum at an interior point $c \in (a, b)$, then $f'(c)$=0.*

According to these theorems, the minimum value of $M(l, s)$ occurs either at a critical point (where $\frac{\partial M}{\partial l}$=0 and $\frac{\partial M}{\partial s}$=0)

Algorithm 1: Checkpointing Strategy with MA-GC (Forward)

Input : Input sequence $\{x_i\}_{i=1}^{S}$, Total layers L, Intervals l_{int}, s_{int}
Output: Checkpoints L_{ckpt}, S_{ckpt}

Initialization: $L_{\text{ckpt}} \leftarrow$ `defaultdict(list)`, $S_{\text{ckpt}} \leftarrow$ `defaultdict(list)`;
for $i \leftarrow 1$ **to** S **do**
  $x \leftarrow x_i$;
  $h \leftarrow \mathbf{0}$;
  for $j \leftarrow 1$ **to** L **do**
    if $j \bmod l_{int} = 0$ **then**
      Append x to $L_{\text{ckpt}}[\lceil i/s_{\text{int}} \rceil, \lceil j/l_{\text{int}} \rceil]$;
    end if
    if $i \bmod s_{int} = 0$ **then**
      Append h to $S_{\text{ckpt}}[\lceil i/s_{\text{int}} \rceil, \lceil j/l_{\text{int}} \rceil]$;
    end if
    $(x, h) \leftarrow \text{Layer}_j(x, h)$;
  end for
end for
return L_{ckpt}, S_{ckpt}

Algorithm 2: Backpropagation with Grid-Cell Restoration on MA-GC (Backward)

Input : Gradients ∇_y, ∇_{state}, Checkpoints L_{ckpt}, S_{ckpt}, Total layers L, Intervals l_{int}, s_{int}

Initialization: $\nabla_h[j] \leftarrow \nabla_{\text{state}}[(j-1) \cdot l_{\text{int}} : j \cdot l_{\text{int}}]$ for each $j = 1, \ldots, \lceil L/l_{\text{int}} \rceil$;
for $i_{blk} \leftarrow S$ **to** 1 **by** $-s_{int}$ **do**
  $\nabla_x \leftarrow \nabla_y[i_{\text{blk}} - s_{\text{int}} : i_{\text{blk}}]$
  for $j_{blk} \leftarrow L$ **to** 1 **by** $-l_{int}$ **do**
    $i_{\text{idx}} \leftarrow \lceil i_{\text{blk}}/s_{\text{int}} \rceil$;
    $j_{\text{idx}} \leftarrow \lceil j_{\text{blk}}/l_{\text{int}} \rceil$;
    $x_{\text{ckpt}} \leftarrow L_{\text{ckpt}}[i_{\text{idx}}, j_{\text{idx}}]$;
    $h_{\text{ckpt}} \leftarrow S_{\text{ckpt}}[i_{\text{idx}}, j_{\text{idx}}]$;
    $(x, h) \leftarrow$ Recompute_Forward$(x_{\text{ckpt}}, h_{\text{ckpt}}, i_{\text{blk}}, j_{\text{blk}})$;
    $(\nabla_x, \nabla_h[j_{\text{idx}}]) \leftarrow$ Backward$(x, h, \nabla_x, \nabla_h[j_{\text{idx}}])$;
  end for
end for

or at the boundary of the region defined by $1 \leq l \leq L$ and $1 \leq s \leq S$. First, we calculate the partial derivatives of M:

$$\frac{\partial M}{\partial l} = -\frac{LS}{l^2} + s = 0, \tag{5}$$

$$\frac{\partial M}{\partial s} = -\frac{LS}{s^2} + l = 0. \tag{6}$$

Solving these, we find that the critical point occurs at $l{=}s{=}\sqrt[3]{LS}$, with the corresponding minimal memory requirement M^*_{critical}:

$$M^*_{\text{critical}} = 3(LS)^{\frac{2}{3}}. \tag{7}$$

However, since l and s are integers within $1 \leq l \leq L$ and $1 \leq s \leq S$, we should evaluate boundary cases. To do so, we use the **Arithmetic Mean-Geometric Mean (AM-GM) inequality** to derive upper bounds for the minimum values of l and s. The AM-GM inequality states that for non-negative real numbers a and b, their arithmetic mean is at least their geometric mean: $\frac{a+b}{2} \geq \sqrt{ab}$. We can express lower bound of each bound as:

$$M^*_{l\text{-bound}} = S + L\left(\frac{S}{s} + s\right) \geq S + 2L\sqrt{S}, \tag{8}$$

$$M^*_{s\text{-bound}} = L + S\left(\frac{L}{l} + l\right) \geq L + 2S\sqrt{L}. \tag{9}$$

Then, the overall optimized memory M^* is selected based on the regions where each configuration achieves balance according to the AM-GM inequality, as follows:

$$M^* = \begin{cases} \Theta\big((LS)^{\frac{2}{3}}\big) & \text{if } L \leq S^2 \text{ and } S \leq L^2, \\ \Theta(S) & \text{if } L^2 \leq S, \\ \Theta(L) & \text{if } S^2 \leq L. \end{cases} \tag{10}$$

Since we are interested in training longer sequences with a fixed number of layers (*i.e.*, $L < S$), we focus on the region where $L^2 \leq S$. Thus, the memory required simplifies to $M^*{=}\Theta(S)$, and the memory savings ratio $\frac{LS}{M^*}$ is given by:

$$\frac{LS}{M^*} = \Theta(L). \tag{11}$$

Thus, as the sequence length S grows, the upper bound on memory savings achieved by MA-GC scales proportionally to the number of layers L. This analysis demonstrates that MA-GC effectively reduces space complexity from $\Theta(LS)$ to $\Theta(S)$, enabling the processing of very long sequences with significantly reduced memory constraints.

As shown in Fig. 1, our experimental results validate this theoretical analysis. Without gradient checkpointing, the activation memory requirement at sequence length 2^{14} is 42.6 GB, whereas with MA-GC, only 42.2 GB is required at sequence length 2^{19}. This demonstrates the practical effectiveness of MA-GC in drastically reducing memory usage while enabling to handle extremely long sequences.

3.4. Model Architecture

Now, we move on to training Video-Ma^{2}mba with long video data using the proposed MA-GC strategy. Analogous to the widely adopted LMM architecture [25, 26],

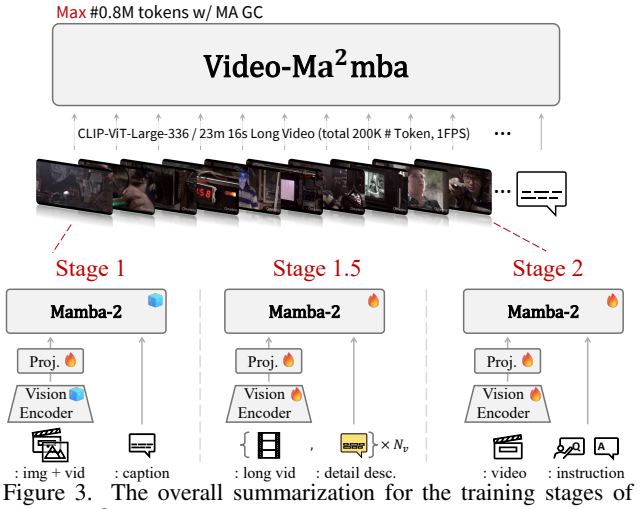


Figure 3. The overall summarization for the training stages of Video-Ma^{2}mba.

as outlined in Fig. 3, Video-Ma^{2}mba follows three main structural components: (i) vision encoder to process input video frames, (ii) cross-modal projector to align vision-text modalities, and (iii) LLM backbone with Mamba-2 architecture (370M / 1.3B / 2.7B).

To ensure thorough coverage of the video content, we systematically sample each frame at 1 FPS. We employed `CLIP-ViT-L-336px` [36] as our vision encoder, extracting features from the penultimate layer, followed by 2x2 bilinear pooling to produce 144 visual tokens per frame. Then, we transform these features to align with the language model's embedding space using a lightweight 2-layer MLP projection with GELU activation. For the language model backbone, we use the Mamba-2 structure instead of standard Transformer-based LLMs, enabling more efficient embedding process with newly designed MA-GC strategy.

3.5. Training Stages

Our training pipeline consists of three stages as summarized in Fig. 3. Beyond the conventional two step training steps for LMMs: alignment training + supervised fine-tuning with instruction data, Li *et al.* [27] have highlighted the importance of high-quality knowledge acquisition between these two stages (thus, stage 1.5). By exploiting the extended context length with the MA-GC during training, we reemphasize that our primary goal in training Video-Ma^{2}mba is to enhance the model's ability to process and learn from long-form video data effectively. Here, expanding the stage 1.5 learning [27], we train Video-Ma^{2}mba using an interleaved learning approach with long video data [19], which comprises densely captioned video-text pairs covering entire long-sequence videos, each segment described in detail. By doing so, Video-Ma^{2}mba can preserve the narrative flow across video segments, enhancing its temporal understanding by learning the sequential relationships within the video content. Through the refined training approach, we aim to develop a robust capability in Video-Ma^{2}mba to handle complex, long-form video data in a contextually aware manner.

Stage 1: Cross-modal Alignment. During the initial step of training our framework, we utilize a dataset comprising a total of 790K image and video pairs with associated texts to ensure cross-modality consistency: 558K image-text pairs, filtered by LLaVA [26], and the remaining video-text pairs sampled from WebVid-2.5M [2]. During this stage, we only optimize the parameters in the projector layer.

Stage 1.5: Long Video Knowledge Learning. As the intermediate long video parametric knowledge learning step, we utilize the SceneWalk dataset [19] that consists of 11.8Khrs of YouTube videos from diverse categories (total 87.8K videos). This dataset includes 1.3M segmented video clips, each with a corresponding scene-level detailed description. We train our model on the next-word generation task by applying an interleaved format to the segmented videos and their corresponding dense captions within each long video instance. In this stage, we unfreeze all parameters to facilitate comprehensive long video understanding.

Stage 2: Supervised Fine-Tuning. We fine-tune our model on a diverse video QA dataset, including LLaVA-Video-178K [49], NeXT-QA [44], ActivityNetQA [46], and PerceptionTest [34]. Together, these sources provide a total of 1.3 million video-instruction QA data —caption entries, open-ended QA, and multiple-choice QA. During this stage, we fine-tune all parameters by unfreezing every network component to enhance the model's QA capabilities.

4. Experiments

4.1. Experimental Setup

Training Details. To train our models, we used 1 node of 8 NVIDIA A100 GPUs, each with 80GB of memory. A cosine learning rate schedule was employed, with a learning rate of 1×10^{-3} for Stage 1 and 4×10^{-5} for both Stage 1.5 and Stage 2. We trained our models for one epoch at each step, with the entire training process taking approximately 4 days to complete for the 3.1B size model. All stages utilized BF16 precision, and we did not employ any form of parameter-efficient fine-tuning such as LoRA [17]. To reduce the memory overhead from optimizer states, we utilized ZeRO-1 optimization [37], enabling more efficient memory management during training. Additional hyper-parameter details are provided in Appendix B.

Implementation Details. Our model, Video-Ma^{2}mba, uses `CLIP-ViT-L-336px` ($\approx$ 0.4B params) vision encoder paired with backbone LMs (370M / 1.3B / 2.7B), resulting in total model sizes of approximately 0.7B, 1.8B, and 3.1B, respectively. For the largest model configuration (3.1B), the backbone Mamba2-2.7B configuration specifically uses an

Model	Size	Video-MME			
		Short	Medium	Long	Avg.
GPT-4V [32]	-	70.5	55.8	53.5	59.9
GPT-4o [33]	-	80.0	70.3	65.3	71.9
Gemini 1.5 Pro [38]	-	81.7	74.3	67.4	75.0
ST-LLM [28]	7B	45.7	36.8	31.3	37.9
VideoChat2-Mistral [21]	7B	48.3	37.0	33.2	39.5
Video-LLaVA [23]	7B	45.3	38.0	36.2	39.9
ShareGPT4Video [4]	8B	48.3	36.3	35.0	39.9
Chat-UniVi-V1.5 [18]	7B	45.7	40.3	35.8	40.6
Qwen-VL-Chat [1]	7B	46.9	38.7	37.8	41.1
SliME [50]	8B	53.3	**42.7**	**39.8**	**45.3**
Video-Ma^{2}mba-0.7B	0.7B	37.4	35.0	26.8	33.1
Video-Ma^{2}mba-1.8B	1.8B	49.4	39.2	31.9	40.3
Video-Ma^{2}mba-3.1B	3.1B	**57.6**	**42.7**	35.4	45.2

(a) Experimental results on Video-MME

Model	Size	LongVideoBench					
		8-15s	15-60s	180-600s	900-3600s	test set	val set
GPT-4o [33]	-	71.6	76.8	66.7	61.6	66.7	66.7
Gemini 1.5 Pro [38]	-	70.2	75.3	65.0	59.1	64.4	64.0
GPT-4-Turbo [31]	-	66.4	71.1	61.7	54.5	60.7	59.1
VideoChat2 [21]	7B	38.1	40.5	33.5	33.6	35.1	36.0
VideoLLaVA [23]	8B	43.1	44.6	36.4	34.4	37.6	39.1
PLLaVA [45]	7B	45.3	47.3	38.5	35.2	39.2	40.2
LLaVA-1.5 [25]	7B	45.0	47.4	40.1	37.0	40.4	40.3
ShareGPT4Video [4]	7B	46.9	50.1	40.0	**38.7**	41.8	39.7
Video-Ma^{2}mba-0.7B	0.7B	43.3	45.4	33.3	28.5	34.2	34.0
Video-Ma^{2}mba-1.8B	1.8B	48.4	49.5	39.6	34.1	39.8	38.0
Video-Ma^{2}mba-3.1B	3.1B	**55.4**	**55.6**	**42.4**	38.5	**44.2**	**43.0**

(b) Experimental results on LongVideoBench

Table 1. Performance comparison across video length categories in Video-MME and LongVideoBench benchmarks.

embedding dimension of 2560, 64 layers, and a vocabulary size of 50,277. The proposed MA-GC is applied throughout all training stages to maximize memory efficiency, enabling our model to handle a maximum sequence length of 0.8M (approx. 1.5-hrs) during training, and generate responses with input sequences up to 2M (approx. 4-hrs) with an 80GB VRAM size GPU.

Memory-Efficient Setup in MA-GC. For the MA-GC setup, we aim to minimize memory usage by selecting optimal intervals l and s for layer-wise and sequence-wise checkpoints, respectively, within the bounds $1 \leq l \leq L$ and $1 \leq s \leq S$. These parameters are chosen to reduce the total memory requirement, as expressed in Eq. (4), covering layer and sequence checkpoints and grid checkpoint cells. Algorithm 1 manages the forward pass by storing checkpoints L_{ckpt} and S_{ckpt} at defined intervals, while Algorithm 2 restores these activations during backpropagation, further optimization. For efficiency in SSD [11], we restrict s to multiples of 256 when $S \geq 256$, which helps maintain high processing performance in the SSD's state scan logic. This approach, illustrated in Fig. 2, enables memory-efficient handling of long sequence lengths, significantly

Model	Size	ActNet-QA		VCG	MVBench
		Acc.	Score	Acc.	Acc.
GPT4V [32]	-	57.0	-	4.06	43.5
GPT-4o [33]	-	61.9	-	-	-
Gemini 1.5 Pro [38]	-	57.5	-	-	-
VideoLLaMA [47]	7B	12.4	1.1	2.16	34.1
Video-ChatGPT [29]	7B	35.2	2.7	2.42	32.7
MovieChat [39]	7B	45.7	-	2.67	-
Chat-UniVi [18]	7B	46.1	3.2	2.99	-
LLaMA-VID [22]	7B	47.4	3.3	2.89	41.3
VideoChat2-Mistral [21]	7B	49.1	3.3	2.98	**62.3**
ShareGPT4Video [4]	8B	50.8	-	-	51.2
VideoLLaMA2 [7]	7B	**53.0**	3.3	**3.13**	54.6
Video-Ma^{2}mba-0.7B	0.7B	43.8	3.2	2.69	41.1
Video-Ma^{2}mba-1.8B	1.8B	50.0	3.1	2.76	44.4
Video-Ma^{2}mba-3.1B	3.1B	51.7	**3.4**	3.03	48.3

Table 2. Benchmark results for ActivityNetQA, VideoChatGPT, and MVBench, comparing Video-Ma^{2}mba and baselines.

extending the model's feasible input size during training.

Evaluation Metrics & Setup. We primarily assess our model using two categorized video analysis benchmarks: long video understanding and general video understanding. For long video benchmarks, we employ Video-MME [13] and LongVideoBench [43], both of which include test instances with video durations of up to 2-hours. For shorter, yet more generalized benchmarks, we utilize four different video analysis benchmarks: ActivityNetQA [46], VideoChatGPT [29], and MVBench [21]. We used `gpt-3.5-turbo-0125` to evaluate responses, implementing a termination criterion to handle repetitive sequences, where generation halts if five consecutive tokens previously appeared. Given our primary model, Video-Ma^{2}mba-3.1B, our main comparisons are with baselines that has ∼8B parameters.

4.2. Experimental Results

Results on Long Video Analysis. We report the long video comprehension results using Video-MME [13] and LongVideoBench [43] in Tab. 1. Despite its smaller scale, Video-Ma^{2}mba outperforms most 7B models in both benchmarks. This is due to our method of accessing the entire video at 1-second intervals, contrasting with the sparse frame sampling strategy. Our comprehensive approach facilitates frequent observations, highlighting our framework's effectiveness against larger 7B models. By selectively preserving key information, Video-Ma^{2}mba avoids the typical information loss and computational burdens, delivering precise, context-aware responses for long videos.

Results on General Video Analysis. In Tab. 2, we summarize general video analysis in several benchmarks: ActivityNetQA [46], VideoChatGPT [29], and MVBench [21]. These benchmarks are much shorter than the previously used long video benchmarks, but provide a comprehensive foundation for assessing our model's capability to an-

Method	Model	Sequence Length ($S = 2^n$)											
		10	11	12	13	14	15	16	17	18	19	20	21
GC off : $O(L \cdot S)$	350M, L=48, d=1024	0.9	1.7	3.3	6.6	13.3	26.5	52.9	-	-	-	-	-
	1.3B, L=48, d=2048	1.7	3.3	6.5	13.1	26.0	52.1	-	-	-	-	-	-
	2.7B, L=64, d=2560	2.7	5.4	10.7	21.4	42.6	-	-	-	-	-	-	-
GC on : $O(L \cdot S)$	350M, L=48, d=1024	0.4	0.7	1.3	2.7	5.5	10.9	21.9	43.7	-	-	-	-
	1.3B, L=48, d=2048	0.7	1.4	2.7	5.5	10.9	21.8	43.5	-	-	-	-	-
	2.7B, L=64, d=2560	1.1	2.2	4.4	8.7	17.4	34.7	69.3	-	-	-	-	-
Sqrt GC : $O(\sqrt{L} \cdot S)$	350M, L=48, d=1024	0.2	0.4	0.8	1.6	3.1	6.2	12.3	24.6	49.3	-	-	-
	1.3B, L=48, d=2048	0.4	0.8	1.5	3.1	6.1	12.1	24.3	48.5	-	-	-	-
	2.7B, L=64, d=2560	0.6	1.1	2.2	4.4	8.7	17.3	34.6	69.3	-	-	-	-
MA-GC : $O(S)$	350M, L=48, d=1024	0.3	0.5	0.6	1.1	1.6	2.4	3.8	5.5	8.8	15.4	23.1	40.2
	1.3B, L=48, d=2048	0.5	0.9	1.2	.2.1	3.7	4.8	7.4	11.3	17.7	30.8	45.8	-
	2.7B, L=64, d=2560	0.7	1.1	1.8	2.7	4.2	6.9	10.5	17.2	25.9	42.2	-	-

Table 3. Memory overhead (GB) for GC methods in Mamba-2-2.7B across sequence lengths ($S = 2^n$). "GC off" indicates no checkpointing; "GC on" applies checkpointing per layer; "Sqrt GC" groups layers by $\sqrt{L}$; and "MA-GC" optimizes based on sequence length. Each cell show peak memory during activation and backpropagation (BF16 precision), excluding model weights and gradients.

Tr Stage	Frame Limit		Video-MME			
1/ 1.5 /2	train	infer	Short: ≤2m	Mid: 4-15m	Long: 30-60m	Overall
✓ ✗ ✓	16 frm	8 frm	49.0	38.7	33.8	40.5
		16 frm	**50.0**	**40.7**	**34.6**	**41.7**
✓ ✗ ✓	1 fps	8 frm	47.7	37.9	32.2	39.3
		16 frm	50.6	39.4	33.2	41.1
		32 frm	52.7	40.8	33.9	42.4
		1 fps	**54.4**	**41.4**	**34.4**	**43.4**
✓ ✓ ✓	1 fps	8 frm	53.3	39.3	32.2	41.6
		16 frm	55.9	41.3	33.9	43.7
		32 frm	**57.9**	41.9	33.9	44.6
		1 fps	57.6	**42.7**	**35.4**	**45.2**

Table 4. Ablation study on frame size and Stage 1.5 effects in Video-MME using Video-Ma^{2}mba-3.1B.

alyze videos and answer related questions. As in the table, we also demonstrate competitive performance of Video-Ma^{2}mba across the benchmarks, holding its own against larger models with parameters over 7B.

4.3. Additional Analyses on Video-Ma^{2}mba

Multi-Axis Gradient Checkpointing For each model configuration, we measured peak memory usage at different sequence lengths to evaluate the memory efficiency of the MA-GC setup. Memory measurements were taken in two steps: (i) capturing baseline memory immediately after loading the model, and (ii) recording peak memory during backpropagation. The difference provides the sequence-dependent memory overhead, reflecting the impact of MA-GC on memory savings. Results, shown in Tab. 3, demonstrate a reduction in space complexity from $O(L \cdot S)$ to $O(S)$, highlighting the efficiency gains of MA-GC over standard checkpointing methods.

Effect of Frame Restriction and Stage 1.5. We conduct ablation studies for the effectiveness of frame threshold and long video knowledge learning (stage 1.5). As summarized in Tab. 4, when the model was trained with a 16-frame limit, it showed lower performance across all video lengths compared to the our framework, achieving a +1.7 points (4.1%) improvement. This indicates that limiting frames can cause the model to miss essential visual cues, impacting comprehension. Including stage 1.5 for long video understanding led to consistent gains across all video length, with performance increasing by +1.8 points (4.1%). Notably, while models trained with 16-frame and 1 FPS limits showed minimal difference without stage 1.5, adding of this stage improved performance with a gain of +1.0 points (2.9%) due to enhanced context tracking over extended sequences.

5. Discussion and Conclusion

Discussion. Even if we have achieved promising results in our experiments, there are several discussion points and limitations. The recently launched Mamba-2, unlike established Transformer-based LLMs, is still immature with a smaller model size and insufficient QA capabilities from limited language instruction training, potentially capping performance. Future enhancements should expand the architecture and diversify training to fully exploit Mamba-2's potential, possibly outperforming more mature models.

Additionally, we propose a method of feeding entire lengthy frames, up to 0.8M sequence length, all at once to LM backbones, although this is not the only approach to optimizing performance. In particular, for lengthy videos, retrieving targeted salient information through selective frame sampling may prove to be a more effective modeling strategy and remains an intriguing direction for future research.

Conclusion. In this work, we propose Video-Ma^{2}mba, a novel Large Multi-modal Model designed for efficient long video understanding, which integrates the Mamba structure into the vision modality. In addition, to push the boundary of memory efficiency, we introduce Multi-axis Gradient Checkpointing strategy and achieve significant memory savings, enabling the processing of extended video sequences up to 0.8M context length. Our extensive validation across multiple benchmarks confirms that Video-Ma^{2}mba not only matches but in some cases exceeds the performance of larger models, highlighting the effectiveness and potential of our approach in pushing the boundaries of video sequence modeling.

References

[1] Jinze Bai, Shuai Bai, Shusheng Yang, Shijie Wang, Sinan Tan, Peng Wang, Junyang Lin, Chang Zhou, and Jingren Zhou. Qwen-vl: A versatile vision-language model for understanding, localization, text reading, and beyond. *arXiv preprint arXiv:2308.12966*, 1(2):3, 2023. 7

[2] Max Bain, Arsha Nagrani, Gül Varol, and Andrew Zisserman. Frozen in time: A joint video and image encoder for end-to-end retrieval. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 1728–1738, 2021. 6

[3] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020. 1, 2

[4] Lin Chen, Xilin Wei, Jinsong Li, Xiaoyi Dong, Pan Zhang, Yuhang Zang, Zehui Chen, Haodong Duan, Bin Lin, Zhenyu Tang, et al. Sharegpt4video: Improving video understanding and generation with better captions. *arXiv preprint arXiv:2406.04325*, 2024. 7

[5] Shouyuan Chen, Sherman Wong, Liangjian Chen, and Yuandong Tian. Extending context window of large language models via positional interpolation. *ArXiv*, abs/2306.15595, 2023. 2

[6] Tianqi Chen, Bing Xu, Chiyuan Zhang, and Carlos Guestrin. Training deep nets with sublinear memory cost. *ArXiv*, abs/1604.06174, 2016. 2, 3, 4

[7] Zesen Cheng, Sicong Leng, Hang Zhang, Yifei Xin, Xin Li, Guanzheng Chen, Yongxin Zhu, Wenqi Zhang, Ziyang Luo, Deli Zhao, et al. Videollama 2: Advancing spatial-temporal modeling and audio understanding in video-llms. *arXiv preprint arXiv:2406.07476*, 2024. 7

[8] Wei-Lin Chiang, Zhuohan Li, Zi Lin, Ying Sheng, Zhanghao Wu, Hao Zhang, Lianmin Zheng, Siyuan Zhuang, Yonghao Zhuang, Joseph E. Gonzalez, Ion Stoica, and Eric P. Xing. Vicuna: An open-source chatbot impressing gpt-4 with 90%* chatgpt quality, 2023. 2

[9] Wenliang Dai, Junnan Li, Dongxu Li, Anthony Tiong, Junqi Zhao, Weisheng Wang, Boyang Li, Pascale Fung, and Steven Hoi. InstructBLIP: Towards general-purpose vision-language models with instruction tuning. In *Advances in Neural Information Processing Systems*, 2023. 1

[10] Zihang Dai, Zhilin Yang, Yiming Yang, Jaime G. Carbonell, Quoc V. Le, and Ruslan Salakhutdinov. Transformer-xl: Attentive language models beyond a fixed-length context. In *Annual Meeting of the Association for Computational Linguistics*, 2019. 2

[11] Tri Dao and Albert Gu. Transformers are ssms: Generalized models and efficient algorithms through structured state space duality. *ArXiv*, abs/2405.21060, 2024. 2, 3, 7

[12] Jeffrey L. Elman. Finding structure in time. *Cogn. Sci.*, 14: 179–211, 1990. 3

[13] Chaoyou Fu, Yuhan Dai, Yongdong Luo, Lei Li, Shuhuai Ren, Renrui Zhang, Zihan Wang, Chenyu Zhou, Yunhang Shen, Mengdan Zhang, et al. Video-mme: The first-ever comprehensive evaluation benchmark of multi-modal llms in video analysis. *arXiv preprint arXiv:2405.21075*, 2024. 7, 3

[14] Google. Gemini, 2023. 1

[15] Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. *ArXiv*, abs/2312.00752, 2023. 2, 3

[16] Bo He, Hengduo Li, Young Kyun Jang, Menglin Jia, Xuefei Cao, Ashish Shah, Abhinav Shrivastava, and Ser-Nam Lim. Ma-lmm: Memory-augmented large multimodal model for long-term video understanding. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 13504–13514, 2024. 2, 3

[17] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*, 2021. 6

[18] Peng Jin, Ryuichi Takanobu, Wancai Zhang, Xiaochun Cao, and Li Yuan. Chat-univi: Unified visual representation empowers large language models with image and video understanding. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 13700–13710, 2024. 3, 7

[19] Junho Kim, Hyunjun Kim, Hosu Lee, and Yong Man Ro. Salova: Segment-augmented long video assistant for targeted retrieval and routing in long-form video analysis. *arXiv preprint arXiv:2411.16173*, 2024. 6

[20] Bo Li, Yuanhan Zhang, Dong Guo, Renrui Zhang, Feng Li, Hao Zhang, Kaichen Zhang, Yanwei Li, Ziwei Liu, and Chunyuan Li. Llava-onevision: Easy visual task transfer. *arXiv preprint arXiv:2408.03326*, 2024. 2

[21] Kunchang Li, Yali Wang, Yinan He, Yizhuo Li, Yi Wang, Yi Liu, Zun Wang, Jilan Xu, Guo Chen, Ping Luo, et al. Mvbench: A comprehensive multi-modal video understanding benchmark. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 22195–22206, 2024. 1, 7

[22] Yanwei Li, Chengyao Wang, and Jiaya Jia. Llama-vid: An image is worth 2 tokens in large language models. In *European Conference on Computer Vision*, pages 323–340. Springer, 2025. 3, 7

[23] Bin Lin, Yang Ye, Bin Zhu, Jiaxi Cui, Munan Ning, Peng Jin, and Li Yuan. Video-llava: Learning united visual representation by alignment before projection. *arXiv preprint arXiv:2311.10122*, 2023. 1, 2, 7

[24] Ji Lin, Hongxu Yin, Wei Ping, Pavlo Molchanov, Mohammad Shoeybi, and Song Han. Vila: On pre-training for visual language models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 26689–26699, 2024. 2

[25] Haotian Liu, Chunyuan Li, Yuheng Li, and Yong Jae Lee. Improved baselines with visual instruction tuning. *arXiv preprint arXiv:2310.03744*, 2023. 1, 5, 7

[26] Haotian Liu, Chunyuan Li, Qingyang Wu, and Yong Jae Lee. Visual instruction tuning. In *Advances in Neural Information Processing Systems*, 2023. 1, 5, 6

[27] Haotian Liu, Chunyuan Li, Yuheng Li, Bo Li, Yuanhan Zhang, Sheng Shen, and Yong Jae Lee. Llava-next: Improved reasoning, ocr, and world knowledge, 2024. 6

[28] Ruyang Liu, Chen Li, Haoran Tang, Yixiao Ge, Ying Shan, and Ge Li. St-llm: Large language models are effective temporal learners. In *European Conference on Computer Vision*, pages 1–18. Springer, 2025. 7

[29] Muhammad Maaz, Hanoona Rasheed, Salman Khan, and Fahad Shahbaz Khan. Video-chatgpt: Towards detailed video understanding via large vision and language models. *arXiv preprint arXiv:2306.05424*, 2023. 1, 3, 7

[30] OpenAI. ChatGPT. https://openai.com/blog/chatgpt/, 2023. 1

[31] OpenAI. Gpt-4 technical report, 2023. 1, 7

[32] OpenAI. GPT-4V(ision) System Card, 2023. 7

[33] OpenAI. Hello gpt-4o, 2024. 7

[34] Viorica Patraucean, Lucas Smaira, Ankush Gupta, Adria Recasens, Larisa Markeeva, Dylan Banarse, Skanda Koppula, Mateusz Malinowski, Yi Yang, Carl Doersch, et al. Perception test: A diagnostic benchmark for multimodal video models. *Advances in Neural Information Processing Systems*, 36, 2024. 6, 1

[35] Ofir Press, Noah A. Smith, and Mike Lewis. Train short, test long: Attention with linear biases enables input length extrapolation. *ArXiv*, abs/2108.12409, 2021. 2

[36] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, et al. Learning transferable visual models from natural language supervision. In *International conference on machine learning*, pages 8748–8763. PMLR, 2021. 6

[37] Samyam Rajbhandari, Jeff Rasley, Olatunji Ruwase, and Yuxiong He. Zero: Memory optimizations toward training trillion parameter models. In *SC20: International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 1–16. IEEE, 2020. 6

[38] Machel Reid, Nikolay Savinov, Denis Teplyashin, Dmitry Lepikhin, Timothy Lillicrap, Jean-baptiste Alayrac, Radu Soricut, Angeliki Lazaridou, Orhan Firat, Julian Schrittwieser, et al. Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context. *arXiv preprint arXiv:2403.05530*, 2024. 7

[39] Enxin Song, Wenhao Chai, Guanhong Wang, Yucheng Zhang, Haoyang Zhou, Feiyang Wu, Haozhe Chi, Xun Guo, Tian Ye, Yanting Zhang, et al. Moviechat: From dense token to sparse memory for long video understanding. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 18221–18232, 2024. 2, 3, 7

[40] Jianlin Su, Yu Lu, Shengfeng Pan, Bo Wen, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding. *ArXiv*, abs/2104.09864, 2021. 2

[41] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023. 1, 2

[42] Ashish Vaswani, Noam M. Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Neural Information Processing Systems*, 2017. 1, 2

[43] Haoning Wu, Dongxu Li, Bei Chen, and Junnan Li. Longvideobench: A benchmark for long-context interleaved video-language understanding. *arXiv preprint arXiv:2407.15754*, 2024. 7, 4

[44] Junbin Xiao, Xindi Shang, Angela Yao, and Tat-Seng Chua. Next-qa: Next phase of question-answering to explaining temporal actions. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 9777–9786, 2021. 6, 1

[45] Lin Xu, Yilin Zhao, Daquan Zhou, Zhijie Lin, See Kiong Ng, and Jiashi Feng. Pllava: Parameter-free llava extension from images to videos for video dense captioning. *arXiv preprint arXiv:2404.16994*, 2024. 7

[46] Zhou Yu, Dejing Xu, Jun Yu, Ting Yu, Zhou Zhao, Yueting Zhuang, and Dacheng Tao. Activitynet-qa: A dataset for understanding complex web videos via question answering. In *Proceedings of the AAAI Conference on Artificial Intelligence*, pages 9127–9134, 2019. 6, 7, 1

[47] Hang Zhang, Xin Li, and Lidong Bing. Video-llama: An instruction-tuned audio-visual language model for video understanding. *arXiv preprint arXiv:2306.02858*, 2023. 7

[48] Peiyuan Zhang, Kaichen Zhang, Bo Li, Guangtao Zeng, Jingkang Yang, Yuanhan Zhang, Ziyue Wang, Haoran Tan, Chunyuan Li, and Ziwei Liu. Long context transfer from language to vision. *arXiv preprint arXiv:2406.16852*, 2024. 2

[49] Yuanhan Zhang, Jinming Wu, Wei Li, Bo Li, Zejun Ma, Ziwei Liu, and Chunyuan Li. Video instruction tuning with synthetic data. *arXiv preprint arXiv:2410.02713*, 2024. 6, 1

[50] Yi-Fan Zhang, Qingsong Wen, Chaoyou Fu, Xue Wang, Zhang Zhang, Liang Wang, and Rong Jin. Beyond llava-hd: Diving into high-resolution large multimodal models. *arXiv preprint arXiv:2406.08487*, 2024. 3, 7

Look Every Frame All at Once: Video-Ma^{2}mba for Efficient Long-form Video Understanding with Multi-Axis Gradient Checkpointing

Supplementary Material

A. Interaction Schema

Video-Ma^{2}mba processes entire video information sequentially at 1 FPS, formulating answers to user queries while retaining relevant details for subsequent questions. Unlike attention-based models, this sequence-modeling approach relies on its ability to remember and retrieve crucial details for future unseen queries. The interaction style is structured in below Tab. 5.

Video QA Instruction:

```
<|system|>You are a helpful
assistant.<|endoftext|>
<|user|>
  <frame1_1><frame1_2> ··· <frame1_144>
  <frame2_1><frame2_2> ··· <frame2_144>
  ...
  <frameN_1><frameN_2> ··· <frameN_144>
(question_1)<|endoftext|>
<|assistant|>(response_1)<|endoftext|>
<|user|>(question_2)<|endoftext|>
<|assistant|>(response_2)<|endoftext|>
...
```

Table 5. Illustration of the interaction schema of Video-Ma^{2}mba during Stage 2 of SFT. Frames are processed in sequence to generate responses to user instructions, ensuring continuity across queries.

B. Training Details

Data Implementation for Efficient Training. To improve the efficiency of training on large-scale video QA datasets due to academic budgets, we deploy a data compression strategy that groups related QA pairs for the same video into a single sample (*i.e.,* single series of QA sets). This drastically reduces the number of training samples from 1.3M to 184K (*e.g.,* LLaVA-Video-178K [49], NeXT-QA [44], ActivityNetQA [46], PerceptionTest [34]) while preserving the diversity and contextual information of the original dataset. By significantly minimizing decoding overhead, this strategy enables more efficient model training without compromising data diversity.

Training Hyperparameters. All Video-Ma^{2}mba variations are trained under consistent configurations, with slight differences in *per-device batch size* due to hardware constraints. To match the *global batch size* across model variations, we utilize *gradient accumulation*, ensuring a similar training schedule for all variations. The hyperparameters for each training stage are detailed in Tab. 6.

Batch Size and Gradient Accumulation. Each model variation leverages the maximum available GPU memory to determine its per-device batch size. Gradient accumulation is applied to align the global batch size across variations, enabling consistent optimization behavior despite hardware differences.

Training Precision and Gradient Checkpointing. We use BFloat16 precision for all stages and adopt *Multi-Axis Gradient Checkpointing*. This approach significantly reduces memory consumption, enabling training with longer sequences and occasionally accommodating larger batch sizes.

config	Stage1	Stage1.5	Stage2
input modality	Vid + Img	Video	Video
FPS for video		1 FPS	
input resolution		336x336	
trainable params	Projector	Full Model	Full Model
LLM lr	1e-3	4e-5	4e-5
Vision lr	-	4e-6	4e-6
lr scheduler		Cosine Decay	
optimizer		AdamW ($\beta_1 = 0.9, \beta_2 = 0.95$)	
global batch size	512	32	32
train epochs	2	2	2
warmup ratio		0.1	
weight decay		0.05	
gradient clipping		1.0	
training precision		BFloat16	
DeepSpeed stage		ZeRO-1	
GC		Multi-Axis Gradient Checkpointing	

Table 6. Hyperparameters for Training Stages.

C. Memory Estimation Logic

The memory estimation logic in Video-Ma^{2}mba optimizes memory requirements by determining the ideal checkpointing intervals l and s used during forward and backward passes. The constants $C_{S\text{-ckpt}}$, $C_{L\text{-ckpt}}$, C_{grid}, and C_{state} depend on the backbone model's configuration, including SSM implementation, block design, and precision type.

Accordingly, the total expected memory M can be computed as follows:

$$M = M_{L\text{-ckpt}} + M_{S\text{-ckpt}} + M_{\text{grid}} + M_{\text{state}}, \qquad (12)$$

where the each memory component in the above equation

are defined as:

$$M_{L\text{-ckpt}} = \frac{LS}{l} \cdot C_{L\text{-ckpt}}, \tag{13}$$

$$M_{S\text{-ckpt}} = \frac{LS}{s} \cdot C_{S\text{-ckpt}}, \tag{14}$$

$$M_{\text{grid}} = ls \cdot C_{\text{grid}}, \tag{15}$$

$$M_{\text{state}} = s \cdot C_{\text{state}}. \tag{16}$$

C.1. Model-Specific Constants

Tab. 7 outlines the constants for three backbone configurations of Mamba-2 in the BFloat16 precision setting. Note that SSM states use Float32 precision, affecting C_{state} and $C_{S\text{-ckpt}}$ depending on whether half- or single-precision is used.

Model	$C_{L\text{-ckpt}}$	$C_{S\text{-ckpt}}$	C_{grid}	C_{state}
Mamba-2-370m	1,024	269,056	6,432	264,448
Mamba-2-1.3b	2,048	537,344	12,608	528,640
Mamba-2-2.7b	2,560	671,488	15,696	660,736

Table 7. Model-specific constants for memory estimation under BFloat16 precision. Constants reflect relative element counts, where SSM states in Float32 are equivalent to two BFloat16 elements.

Recomputation for Memory Optimization. The term M_{state}, omitted in Eq. (4), arises in Mamba-2 [11] due to recomputation of SSM states during the backward pass. This recomputation reduces memory usage by avoiding the storage of intermediate states during forward computation:

> *"The intermediate states are not stored but recomputed in the backward pass when the inputs are loaded from HBM to SRAM. As a result, the fused selective scan layer has the same memory requirements as an optimized transformer implementation with FlashAttention."* [15]

Although M_{state} grows linearly with s (*i.e.,* $M_{\text{state}}{=}O(s)$), the memory term M_{grid} grows as the product of $l \cdot s$ (*i.e.,* $M_{\text{grid}}{=}O(ls)$), making it asymptotically dominant. Consequently, M_{state} becomes a negligible term in the overall memory complexity, and the analysis in Sec. 3.3 remains valid.

Checkpointing Trade-Offs. Selecting the optimal values for l and s is critical to minimizing M. For example, larger values of s reduce $M_{S\text{-ckpt}}$ but increase restoration overhead during the backward pass (M_{grid} and M_{state}). As shown in Tab. 7, a careful balance for memory savings is required to process long video sequences efficiently.

D. Gradient Checkpointing Time Analysis

We analyze the computational efficiency of various gradient checkpointing methods, with results summarized in Tab. 8. Throughput (tokens per second) indicates the processing speed, while per-token processing time (milliseconds per token) provides a more detailed perspective on computational overhead. The reported measurements represent the median of six runs conducted on an A100 80GB GPU using the Mamba-2-2.7b, which is the backbone of Video-Ma^{2}mba-3.1B. A warm-start configuration was used to minimize initialization overhead, and the times include both the forward and backward computation steps.

Method	**Throughput ↑ (tokens/s)**	**Processing Time ↓ (ms/token)**
GC off @ 2^{14}	12,167.86	0.082
GC on @ 2^{16}	8,449.93	0.118
Sqrt GC @ 2^{17}	8,617.66	0.116
MA-GC @ 2^{19}	7,913.58	0.126

Table 8. Computational analysis of throughput and per-token processing time among gradient checkpointing methods. Results are measured using the Mamba-2-2.7b model on an A100 80GB GPU. The notation **@ 2^n** specifies the sequence length (in tokens) used for measurement.

MA-GC demonstrates the ability to train on sequence lengths up to **32× longer** than the GC-off baseline within the same memory constraints. Although this extended capability comes with a **35% reduction** in throughput, this is a trade-off that enables scalable and efficient training for tasks requiring extremely long sequences.

The *gradient checkpointing* methods can save memory and handle longer sequences but are fundamentally constrained when processing extremely long sequences due to memory limitations. MA-GC overcomes such issues by introducing a multi-axis gradient checkpointing mechanism, enabling training on sequence lengths up to 32× longer while retaining computational feasibility. By efficiently managing memory without incurring prohibitive overhead, MA-GC balances performance and resource efficiency, significantly extending the scalability of large-scale models.

E. Qualitative Evaluation

As in Fig. 4, Fig. 5, and Fig. 6, our analysis provides qualitative outcomes across various benchmarks, clearly demonstrating the adaptability and effectiveness of Video-Ma^{2}mba. Consistent performance across different benchmarks highlights the model's ability to efficiently handle and analyze disparate datasets. This effectiveness stems from Video-Ma^{2}mba's ability to process extensive context using the MA-GC mechanism, which allows it to handle all incoming inputs comprehensively, facilitating robust data processing and insightful reasoning.

Video-MME Results.

Question: How many times does the interviewed girl appear in the video?
Option: (A). 4 / (B). 1 / (C). 2 / (D). 3
Answer: C.
Prediction: C.

Question: Which activity is the most likely one for the two main people in the image?
Option: (A). Having a conversation. / (B). Dancing. / (C). Arguing. / (D). Playing a game.
Answer: B.
Prediction: B.

Question: When does Sion start proxying?
Option: (A). Teammates arrive. / (B). Opponent reaches tower. /
(C). Slaining enemy. / (D). A big wave of minions gets under the tower.
Answer: D.
Prediction: D.

Figure 4. Qualitative examples on Video-MME [13] with Video-Ma^{2}mba-3.1B.

VideoChatGPT Results.

Question: What is the woman doing in the outdoor field during the daytime?
Answer: The woman is participating in a pole vaulting competition.
Prediction: She is running on the track, holding a pole vault pole, and preparing for her pole vault attempt.

Question: What items are on the table in front of the woman at the beginning of the video?
Answer: Tubes and bottles of makeup are on the table in front of the woman.
Prediction: Various makeup items, including a bottle of Olay CC Cream, a tube of foundation, and a lip balm.

Figure 5. Qualitative examples from the Generative Subset of VideoChatGPT [29] with Video-Ma^{2}mba-3.1B.

LongVideoBench Results.

Question: After the female protagonist in the video adds melted butter into the blender containing chocolate cake mix, what happens next in the video?
Option: (A). Mix the two together. / (B). Pour them into a bowl. /
(C). Add more chocolate cake mix. / (D). Add milk. / (E). N/A.
Answer: (A). Mix the two together
Prediction: (A). Mix the two together.

Question: In front of a wooden platform with three potted plants, there is a person looking down with both legs raised on the table. Who is this person?
Option: (A). A woman with brown short hair. / (B). A woman with brown long hair. /
(C). A woman with black short hair. / (D). A woman with black long hair. / (E). N/A.
Answer: B.
Prediction: B.

Question: In the top-right corner of the brown cutting board, there are several pieces of already-cut meat. On the left side of the cutting board, there is uncut meat on a metal plate. In the middle of the screen, a person is holding a pair of tongs. What is this person doing?
Option: (A). Placing the tongs on the cutting board. / (B). Placing the meat from the metal plate onto the cutting board. / (C). Using the tongs to grab vegetables. / (D). Placing the cut meat from the cutting board onto the metal plate. / (E). Putting the tongs into a bowl
Answer: B.
Prediction: B.

Figure 6. Qualitative examples on LongVideoBench [43] with Video-Ma^{2}mba-3.1B.